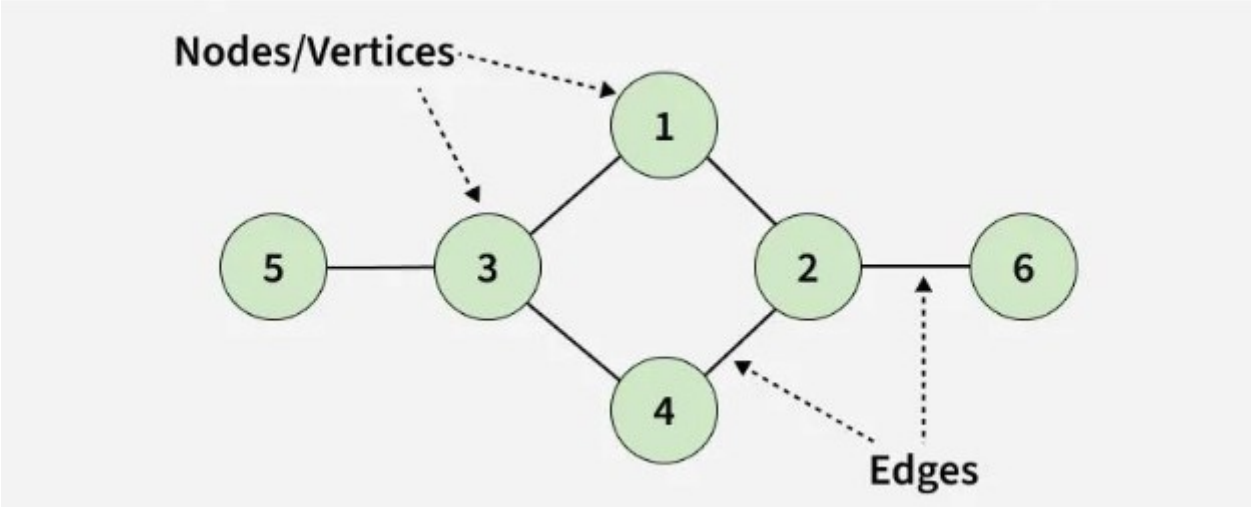
Lecture 13

Digraphs & Recursion



Announcements

- Office hours are up on course website
  - T: 12pm-4pm; F: 11am-1pm
  - Schedule an appointment individually or as a group

- Exam 1 rewrites due Thursday at the beginning of lecture via email
  - Can earn back up to 50% on any question
  - Provide a very thorough answer and write up in a markdown file

Last time

- Depth-First Search (DFS)
- Directed Graphs
- Traversing Directed Graphs

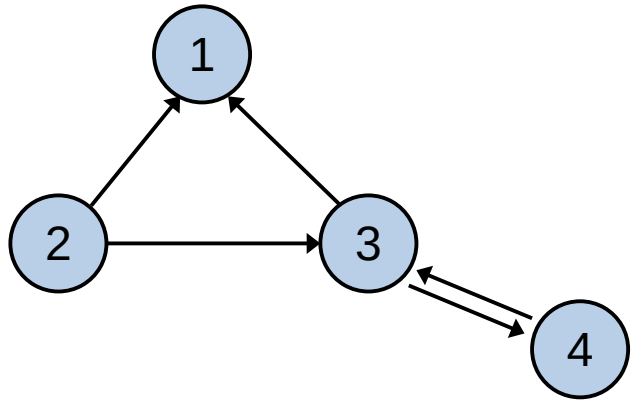
Today

- DAGs
- Topological Sort
- Recursion

Directed Graphs

`G = (V,E)`

- `(u,v) ∈ E` is a directed edge
- We say that u points to v
- `e = (u,v)` leaves u, enters v, is an outgoing edge from u, is an incoming edge to v



Modeling Networks

For each of the following, determine if it a directed or undirected graph fits better:

- Facebook and LinkedIn **Undirected**
- X and Instagram **Directed**
- One-way roads between buildings with a city **Directed**
- Interstate highways connecting cities **Undirected**

Digraph Traversal

Directed Reachability Problem

- Starting from vertex s, can we reach vertex v?
- Finding all vertices reachable from s
- What is the shortest path from s to v?

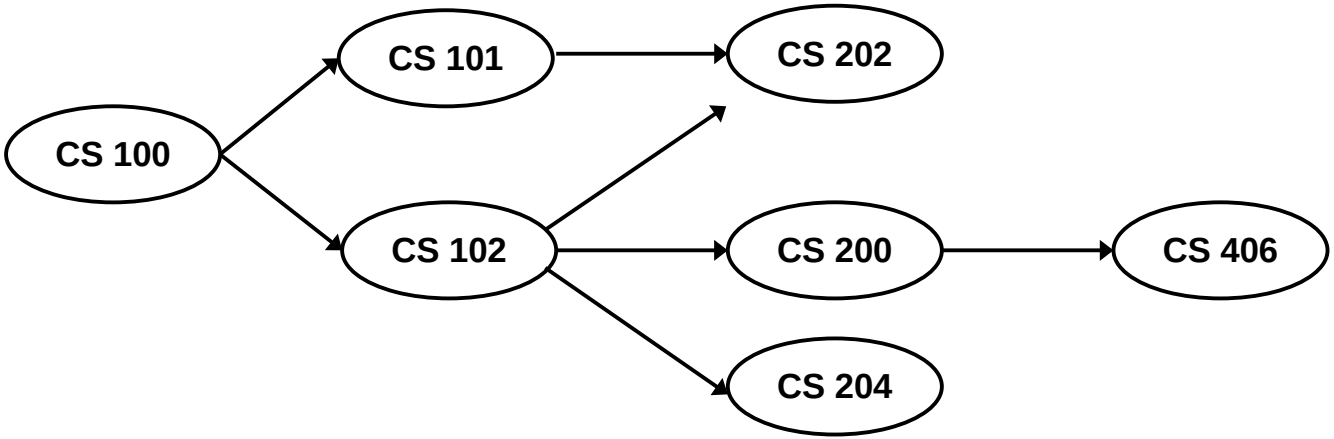
BFS/DFS naturally extend to directed graphs

- Replace "neighbors" with "outgoing neighbors"

Directed Acyclic Graphs

A **DAG** is a directed graph with with no cycles

- Modeling dependencies, e.g., course prerequisites and software requirements



Topological Sorting

A topological sorting is an ordering of the vertices such that all edges "go forward" in the ordering

- A way to schedule classes such that all prerequisites are satisfied
- An order to install packages such that nothing breaks
- Label vertices $v_1, v_2, \ldots, v_n$ such that for every edge $v_i v_j$ we have that $i<j$

Topological Sorting

A topological sorting is an ordering of the vertices such that all edges "go forward" in the ordering

- A way to schedule classes such that all prerequisites are satisfied
- An order to install packages such that nothing breaks
- Label vertices $v_1, v_2, \ldots, v_n$ such that for every edge $v_i v_j$ we have that $i<j$

Is a topological order possible for any directed graph?

Topological Sorting

A topological sorting is an ordering of the vertices such that all edges "go forward" in the ordering

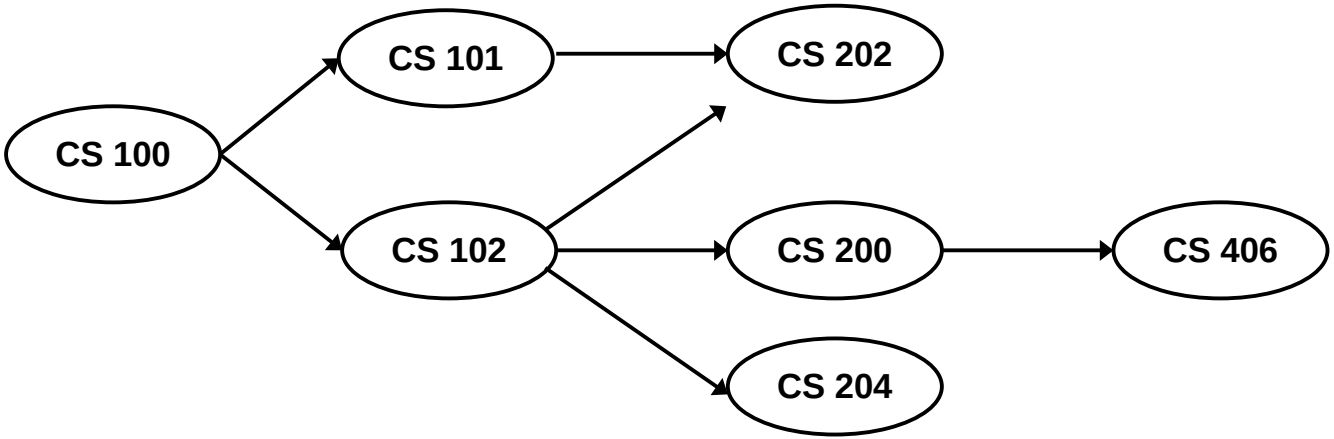
- A way to schedule classes such that all prerequisites are satisfied
- An order to install packages such that nothing breaks
- Label vertices $v_1, v_2, \ldots, v_n$ such that for every edge $v_i v_j$ we have that $i<j$

Is a topological order possible for any directed graph?

- No, a digraph has a topo ordering if and only if it is a DAG

Exercise

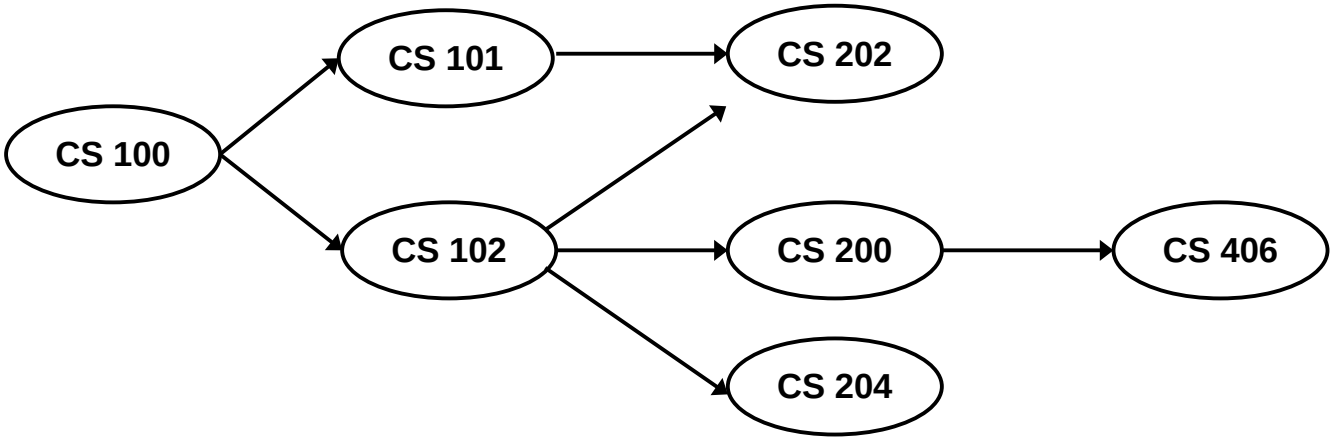
Find a topological order for the following digraph



Exercise

Find a topological order for the following digraph

- 100, 102, 204, 200, 406, 101, 202



Finding a Topo Ordering

Given a DAG `D`, we want to find a topological ordering

Finding a Topo Ordering

Given a DAG `D`, we want to find a topological ordering

while there are vertices remaining:

- find vertex v with no incoming edges
- place v next in the order
- delete v from `D`

Finding a Topo Ordering

Given a DAG `D`, we want to find a topological ordering

while there are vertices remaining:

- find vertex v with no incoming edges
- place v next in the order
- delete v from `D`

Running time: `O(n+m)`

Recursion

Recursive Algorithm

What does it mean for an algorithm to use recursion?

Recursive Algorithm

What does it mean for an algorithm to use recursion?

- The algorithm calls itself (self-referential)
- Hopefully, terminates at some point

Factorial

Let's think about `f(n) = n!`

- `f(4) = 4 * 3 * 2 * 1`

  `= 4 * f(3)`

Factorial

Let's think about `f(n) = n!`

- `f(4) = 4 * 3 * 2 * 1`

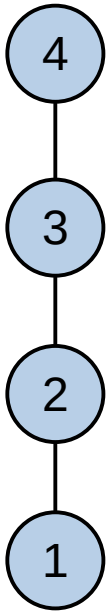
  `= 4 * f(3)`
- `f(n) = n * f(n-1)`
- `f(1) = 1`

Factorial

Let's think about `f(n) = n!`

- `f(4) = 4 * 3 * 2 * 1`

  `= 4 * f(3)`
- `f(n) = n * f(n-1)`
- `f(1) = 1`



Can represent recursion as a tree

Fibonacci Sequence

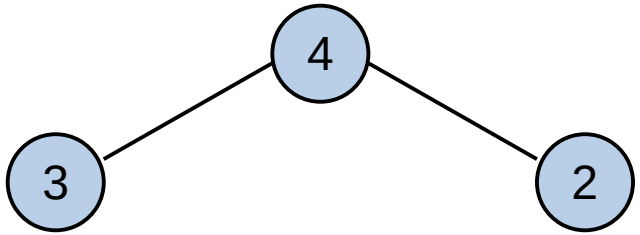
Rules:

- `f(n) = f(n-1) + f(n-2)`
- `f(0) = 0`
- `f(1) = 1`

Fibonacci Sequence

Rules:

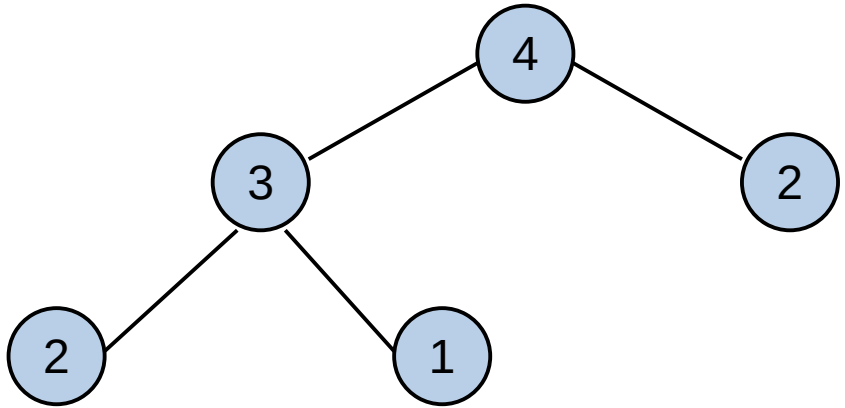
- `f(n) = f(n-1) + f(n-2)`
- `f(0) = 0`
- `f(1) = 1`



Fibonacci Sequence

Rules:

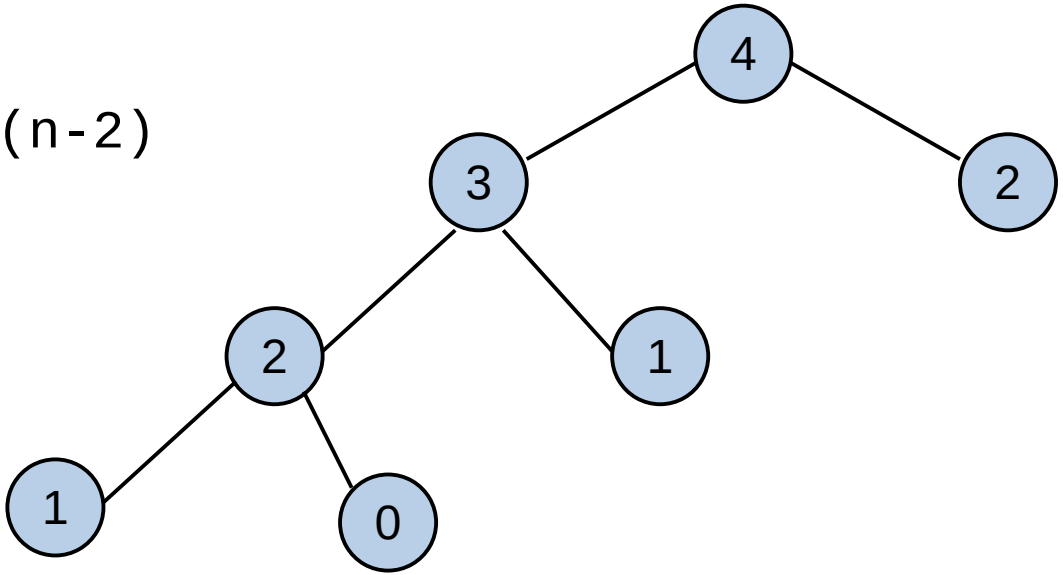
- `f(n) = f(n-1) + f(n-2)`
- `f(0) = 0`
- `f(1) = 1`



Fibonacci Sequence

Rules:

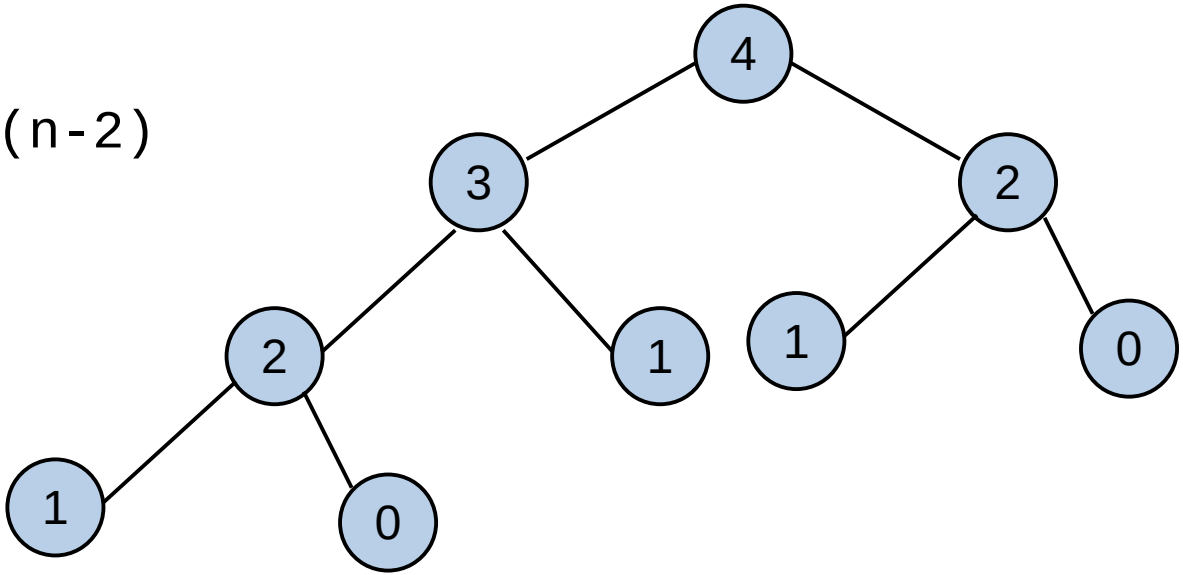
- `f(n) = f(n-1) + f(n-2)`
- `f(0) = 0`
- `f(1) = 1`



Fibonacci Sequence

Rules:

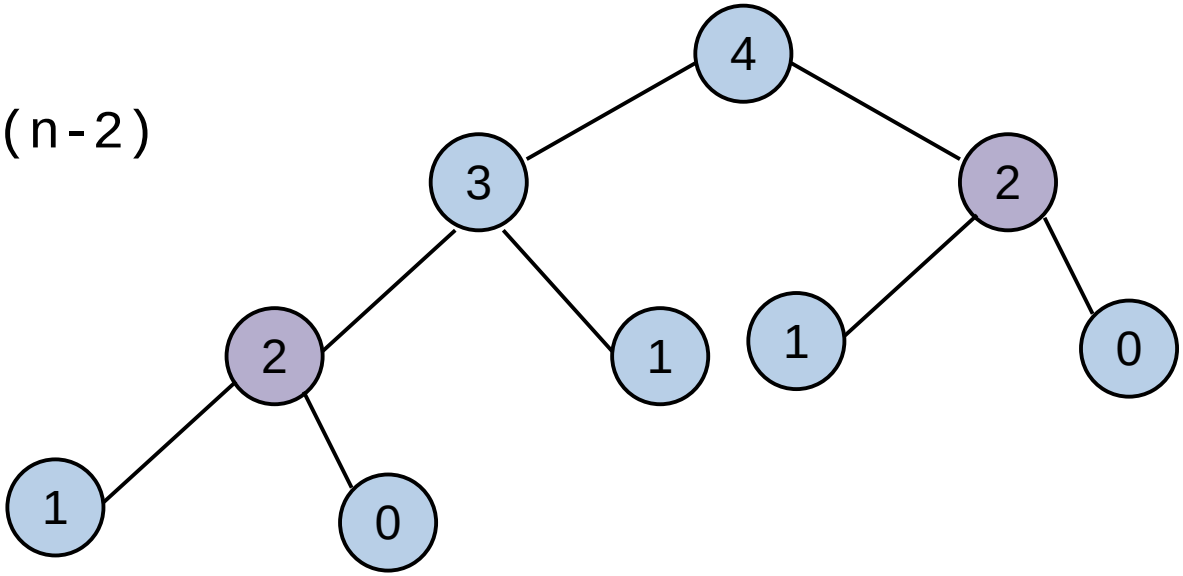
- `f(n) = f(n-1) + f(n-2)`
- `f(0) = 0`
- `f(1) = 1`



Fibonacci Sequence

Rules:

- `f(n) = f(n-1) + f(n-2)`
- `f(0) = 0`
- `f(1) = 1`

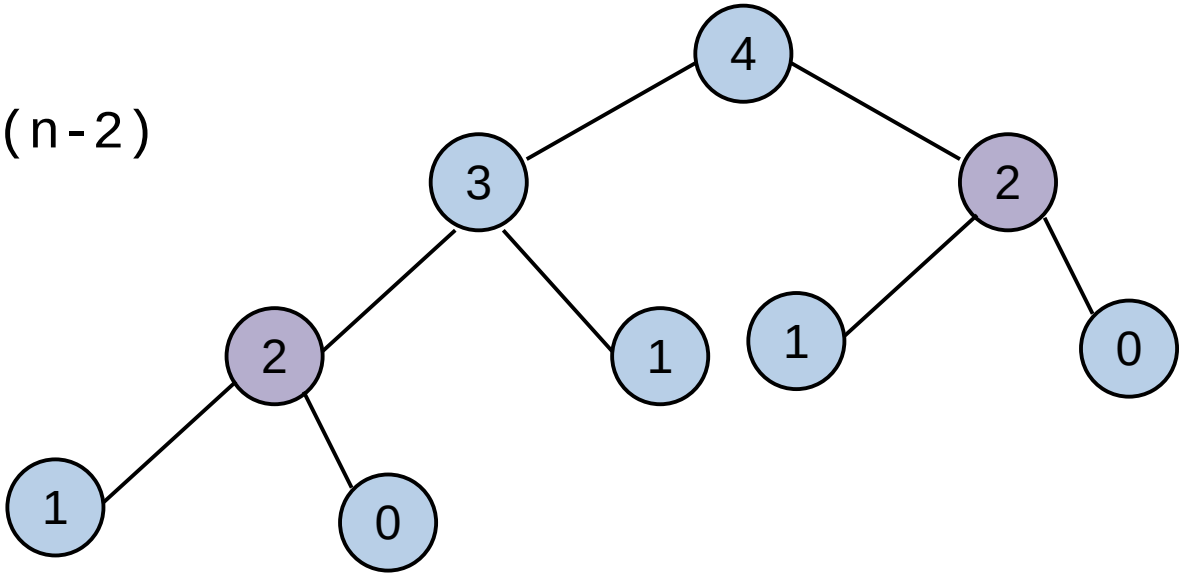


Notice that we're computing some values multiple times

Fibonacci Sequence

Rules:

- `f(n) = f(n-1) + f(n-2)`
- `f(0) = 0`
- `f(1) = 1`



Notice that we're computing some values multiple times

- By computing it once, then caching we can be more efficient (memoization)

Next Time

- Dynamic Programming